

phase1 system design

1. Objective of MVP

Build a system that can:

1. accept user posts + votes
2. compute credibility in real time
3. control visibility (basic propagation)
4. trigger alerts

Without:

- complex ML
- advanced location modeling
- heavy graph computation

2. What to KEEP vs REMOVE

KEEP (Core System)

These are non-negotiable:

User votes Time decay User reliability (simplified) Credibility (Bayesian) Basic anomaly (rate only) Alert condition

REMOVE (For Now)

Do NOT implement initially:

Graph trust propagation (T_i) ML models (C_ML, anomaly ML) Memory similarity Advanced location modeling Device/IP fingerprinting Navigation models

3. Minimal Data Schema

User Table

users (user_id TEXT PRIMARY KEY, alpha FLOAT, beta FLOAT, last_active TIMESTAMP)

Post Table

posts (post_id TEXT PRIMARY KEY, content TEXT, created_at TIMESTAMP, credibility FLOAT, variance FLOAT, interaction_mass FLOAT)

Interaction Table

interactions (id SERIAL, user_id TEXT, post_id TEXT, vote INT, - +1 or -1 timestamp TIMESTAMP)

4. Simplified Mathematics (MVP Version)

4.1 User Reliability

$R_i =$

Update Rule

if vote matches final truth: $\alpha_i += 1$ else: $\beta_i += 1$

4.2 User Weight (Simplified)

$w_i = R_i$

(No anomaly, no experience initially)

4.3 Interaction Mass

$N_j(t) = w_i e^{\{-(t - t_k)\}}$

4.4 Positive / Negative Signals

$S_j^{+,,}; S_j^{-}$

(as defined earlier)

4.5 Credibility (Core Engine)

$C_j(t) =$

4.6 Variance

Use simplified:

$Var_j =$

(No per-interaction loop needed)

4.7 Basic Anomaly (Only Rate)

$Anom_i = 1 - (-)$

(Optional in MVP)

4.8 Alert Condition (Simplified)

$Alert(j) = (C_j > 0.7 ;; N_j > N_{\{min\}})$

5. System Flow (MVP)

User posts content ↓ Other users vote ↓ Store interaction ↓ Update: - S_j^{+}, S_j^{-} - N_j ↓ Compute: - C_j - Var_j ↓ Check: - Alert condition ↓ Display / notify

6. Backend Structure

Core Services

1. Interaction Service

Handles:

POST /vote POST /post

2. Credibility Engine

Runs:

update_post_metrics(post_id)

3. Alert Engine

Runs:

if $C_j > \text{threshold}$ → trigger alert

7. Minimal API Design

Create Post

POST /post { "user_id": "u1", "content": "Fire in building" }

Vote

POST /vote { "user_id": "u2", "post_id": "p1", "vote": 1 }

Get Post

GET /post/:id

Returns:

{ "credibility": 0.82, "variance": 0.1, "status": "trusted" }

8. What This MVP Demonstrates

Even this minimal system proves:

- credibility emerges dynamically
- unreliable users lose influence
- consensus stabilizes truth
- alerts are controlled

9. What You Add NEXT (Phase 2)

After MVP works, add in order:

Phase 2

- time-decayed reliability
- experience score
- better anomaly (entropy + rate)

Phase 3

- graph trust (T_i)
- coordination detection

Phase 4

- location + proximity
- spatial alerts

Phase 5

- ML (content + anomaly)
- memory similarity

10. Critical Insight

Your full system is:

research-grade

Your MVP should be:

proof-of-concept of core mechanism

11. Final Advice

Do NOT try to:

- implement all signals
- tune all hyperparameters
- build full pipeline

Start with:

credibility + votes + time decay

12. One-Line Summary

MVP = weighted voting + Bayesian credibility + threshold alert